Architectural Document: Sprint 2

Project Title: Student Project Collaboration & Teammate Finder

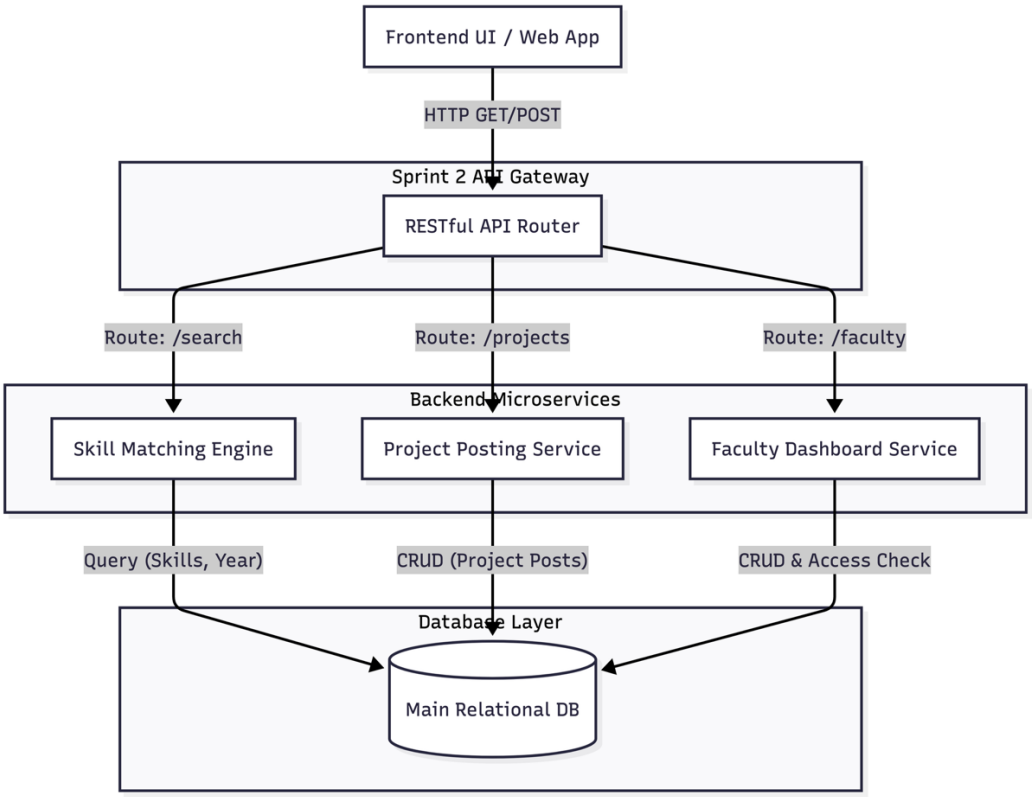
Sprint Focus: Search Engine, Project Posting Board, and Role-Based Portals

1. System Component Architecture Expansion

During Sprint 2, the system's baseline architecture is expanded to transition from basic user management to active networking capabilities. The architecture follows a serverless microservices pattern, ensuring that new features do not disrupt the authentication logic established in Sprint 1.

The core components introduced in this sprint include:

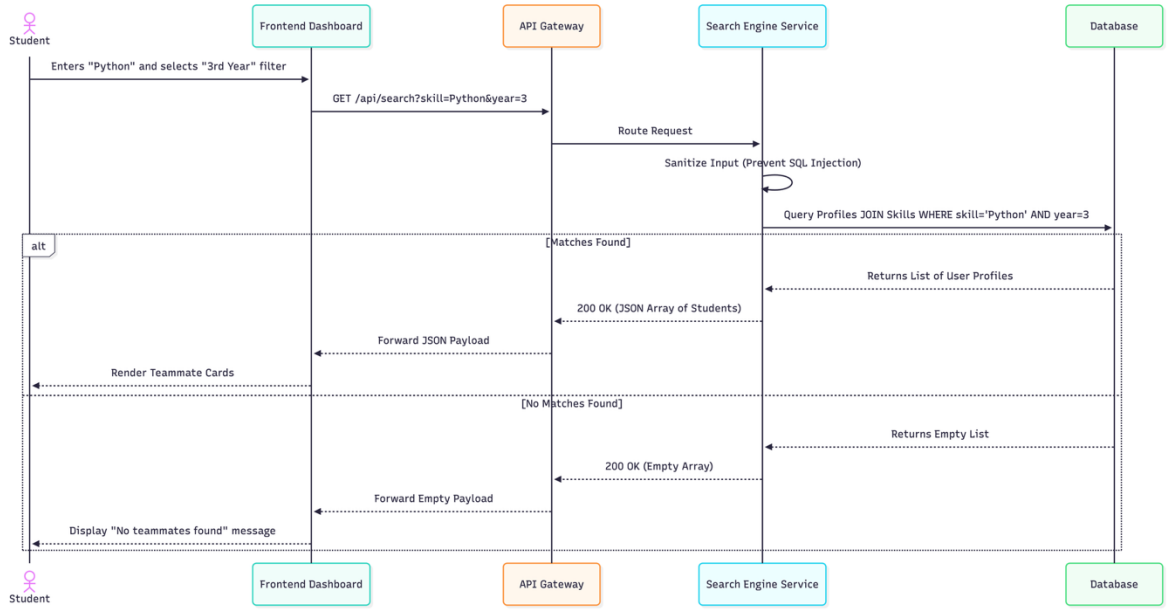
- **RESTful API Gateway:** Acts as the central traffic controller, securely routing frontend HTTP requests (GET, POST, DELETE) to the appropriate backend microservice.
- **Skill Matching Engine:** A dedicated search service that processes user queries, applies filters (such as academic year or department), and retrieves compatible peer profiles from the database.
- **Project Posting Service:** Handles all CRUD (Create, Read, Update, Delete) operations related to the "Teammate Wanted" advertisements on the main student feed.
- **Faculty Dashboard Service:** A protected service module specifically designed to handle the creation and management of research opportunities posted by verified professors.



2. Teammate Search & Discovery Execution Flow

The Search Engine is the most logically complex feature of Sprint 2. The execution flow follows a strict sequence to ensure fast retrieval and data security:

1. **Query Initiation:** The student enters a required skill (e.g., "Python") and applies optional filters (e.g., "3rd Year") via the frontend dashboard.
2. **API Routing:** The frontend transmits this data as a structured HTTP GET request to the API Gateway.
3. **Input Sanitization:** Before interacting with the database, the Skill Matching Engine scrubs the input parameters to prevent SQL injection attacks or malicious payload executions.
4. **Database Query:** The engine executes a relational query, joining the User Profiles table with the Skills table to find exact matches.
5. **Response Delivery:** * *If matches are found:* The engine packages the matched profiles into a JSON array and sends a `200 OK` response back to the frontend to render the teammate cards.
    - *If no matches are found:* The engine returns an empty array, prompting the frontend UI to display a "No teammates found" empty state message.



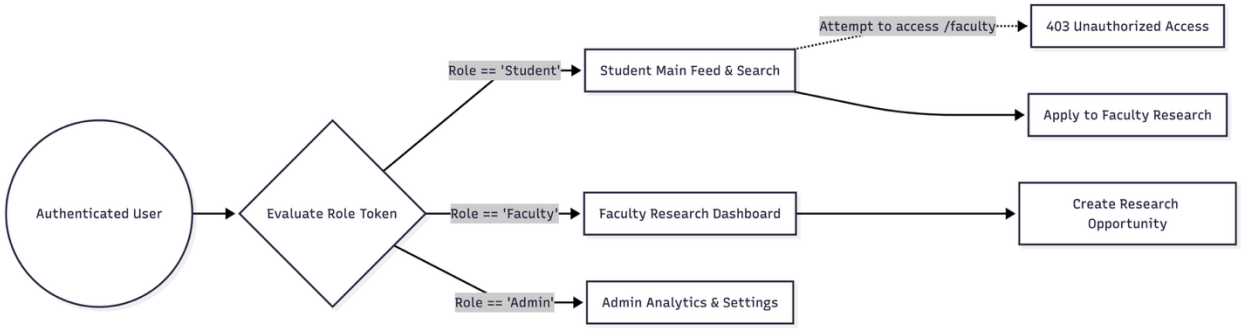
3. API Data Exchange Contracts

To formalize communication between the client and the new serverless functions, the following API contracts have been established. All endpoints utilize lightweight JSON payloads.

Endpoint	HTTP Method	Purpose	Expected Request Payload
/api/search	GET	Fetches students matching specific technical skills and academic filters.	*URL Params:* ?skill=Python&year=3
/api/projects	POST	Creates a new "Teammate Wanted" ad on the main dashboard feed.	{"title": "Hackathon", "role_needed": "Backend"}
/api/projects/{id}	DELETE	Allows a student to remove their own project posting from the feed.	*None (Uses URL Parameter & Auth Token)*
/api/faculty/posts	POST	Allows verified faculty members to publish research opportunities.	{"title": "AI Model", "req_skills": ["Python"]}

4. Role-Based Access Control (RBAC) Protocol

With the introduction of the Faculty Dashboard, strict authorization protocols are enforced at the routing layer to ensure data privacy and system integrity.

When a user authenticates, the system assigns a secure session token containing their specific role (Student, Faculty, or Admin). The RBAC logic dictates the following routing behavior:

- **Student Tokens:** Grants access to the main feed, teammate search, and project posting. If a student attempts to navigate to the /faculty route, the system intercepts the request and throws a 403 Unauthorized Access error.
- **Faculty Tokens:** Bypasses the standard student feed and routes the user directly to the Faculty Research Dashboard, granting exclusive access to the /api/faculty/posts endpoints to recruit research assistants.